

A NOTE FROM PRODUCTION

The demo was the easy part.

Every agent demo works. Then it meets a prompt injection it can't see, an auditor asking “*why did it do that?*”, an eval suite that's really vibes, and a cloud bill that ends the experiment. The loop was never the hard part — **everything around the loop is.**

Injections

input rails that are a regex and good intentions

Audits

decisions nobody recorded, answers nobody has

Evals

vibes as the quality gate

Bills

production-grade only if you can afford it

What belongs between a working loop and a trustworthy agent?



What every agent builder already knows

Frameworks hand you the ReAct loop in fifty lines — and leave guardrails, evaluation, governance, deployment, and the learning curve as exercises for the reader. The tutorials stop exactly where production begins.



So the question this repo answers:

Can one open codebase ship the un-demo-able parts — a guardrail stack with a trained model, an eval pipeline calibrated against humans, policy-gated cloud deployment at hobby cost, and the recipes to rebuild it all — in a form you can verify before you trust it?

THE ANSWER

Yes. It's all here — and checkable.

This repo open-sources the four things agent demos leave out, plus the architecture that holds them together. Six sections follow; every claim carries a file path or a command you can run.

1

Four-layer architecture

12 CI gates; illegal imports fail the build

2

Guardrail ML

fine-tuned DeBERTa-v3, frozen eval gates

3

Goal-judge evals

grounded theory, $\kappa = 1.0$

4

Free web search

\$0 self-hosted SearXNG, no API key

5

GCP deployment

policy-as-code, idles near \$0

6

40+ recipes

rebuild every piece yourself

108K+ lines · 2,500 tests · 0 live LLM calls in CI · built solo in 8 weeks

The trust layer for consequential AI

So the decisions that matter — in finance, healthcare, government — can be handed to software *without handing over accountability*.

Five values — each a constraint, each mapped to the mechanism that makes it verifiable, not merely promised:

I am honest.

Never hide what I did; never call unverified "verified."

BlackBox recorder — a replayable audit trail that captures me regardless of intent.

I take only what is mine.

Use only the data, access, and authority I was granted.

Signed trust kernel + tool validators, allowlists, path sandboxing — held by the build.

I don't grasp or recoil.

A failure is information, not a verdict. Name it plainly.

Input/output guardrails + an evidence-grounded goal-judge that scores work, not confidence.

I stay with the task.

One thing, done with care, then verified. No scope creep.

Four-layer architecture; the test suite fails the build when a boundary is crossed.

I help and share.

Lower the cost of accountable work for those who follow.

Open foundation; externalized, auditable prompts; a read-only explainability surface.

Trust, then verify — the documents state the values; the running system makes them checkable in the code. [MISSION.md](#) · [SOUL.md](#)

Four layers — crossing them fails the build



Every framework draws a layering diagram.



Nothing stops next week's PR from importing across it. Diagrams rot; nobody notices until refactor day.

ORCHESTRATION

LangGraph topology & state — thin wrappers only

COMPONENTS

router · evaluator · plan builder · goal judge — LangGraph imports forbidden

SERVICES

prompts · guardrails · governance · memory · tools · web search

TRUST KERNEL

pure types & crypto — stdlib + Pydantic only, no I/O, no logging

dependencies flow downward only



The boundary is a test,
not a convention

12 architecture-gate modules check dependency rules, service isolation, and layer placement on every commit. 2,500 tests overall — a 2:1 test-to-code ratio, with zero live LLM calls in CI.

Verify: `pytest tests/architecture/ -q`

One task, five phases — not one hopeful loop



A bare ReAct loop is one pass: if the model misreads the task, the task fails.



Production needs a gate before the spend and a quality check after — thinking and checking, not just doing.

1 · INGRESS

`guard_input_node`

Reject unsafe input at Step 0 — before any token spend.

2 · PLANNING

`route_node`

Pick depth (L0–L2); build a deterministic plan floor.

3 · EXECUTION

`call_llm / supervisor`

T0 ReAct spine, or T3 fan-out for independent work.

4 · EVALUATION

`evaluate_node`

GoalJudge scores the result; T2 reflexion if it falls short.

5 · EXIT

`reasoning_recap`

Synthesize a human-readable 'why' for the UI — or escalate.

All five are real nodes in `orchestration/react_loop.py` verify: `grep -n "def .*_node" orchestration/react_loop.py`

Two orthogonal axes, one unbreakable floor

S

Most loops use one strategy for every task — over-planning the easy, under-planning the hard.

C

And when an LLM plan is malformed, a loop with no backstop simply stalls.

	L0 max 1 step	L1 max 3 steps	L2 max 5 steps
T1 plan artifact	Yes	Yes	Yes
T2 reflexion	Yes	Yes	Yes
T3 fan-out	Never	if ≥ 2 steps	if ≥ 2 steps

Depth sets the step budget · tiers add mechanisms — independently.

A

The plan floor never depends on the LLM

`build_plan_artifact` lays down an ordered, deterministic plan first. The LLM only enriches it — if the model returns nothing usable, the floor still runs. No brittle stalls.

T3 default: `t3_fanout_enabled = False`

Parallelism that refuses to corrupt



Fan-out is fast — when tasks are truly independent.



Run coupled steps in parallel (hotel dates that depend on flights) and you corrupt the result, silently.

Depth = L0?



DECLINE

Sequential dependent?



DECLINE • GAIA guard

No plan generator?



DECLINE

validate_independence fails?



DECLINE

All checks pass



FAN_OUT

MEASURED • T3 STRESS CORPUS

1.0 precision

fp = 0 • 10/10 dependent tasks declined

Small N, designed corpus — a discipline that holds on the cases it was tested on, not a population guarantee.

Decline is the default on any unproven path.

[docs/plans/t3_stage_b_case_walkthrough.md](#)

From a brittle loop to a tiered reasoning ladder

S

A single ReAct loop applies one strategy to every task and stalls on a bad plan.

C

Last week: a four-rung ladder that picks effort by need — every rung shadow-first, default OFF, promoted only on evidence.

T1

Plan with a replan gate

PlanGenerator enriches the deterministic floor; rebuilds when a surprising tool result makes the plan stale.

`plan_source`**T2**

Reflexion re-entry

decide_reentry critiques a weak result and loops back — with a D1 attempt ceiling and a D3 prose-thrash backstop.

`reflexion_enabled`**T3**

Supervisor fan-out

plan_delegations decomposes independent work; validate_independence declines anything coupled (GAIA guard).

`t3_fanout_enabled`**\$5**

Escalation predicate

decide_escalation composes the re-entry/continue signals into one decision, measured against a precision oracle.

`escalation_oracle`

Shadow-first by default → prod graph stays byte-identical until a rung earns promotion. `services/base_config.py`

Memory: recall, capture, full audit trail



An agent that forgets every session can't personalize or improve.



This week: durable recall + typed auto-capture — every read, write, and reject leaves a governance carrier.

RECALL

MEMORY_RECALLED

route_node pulls relevant memories above a relevance floor (dedup'd) into the prompt.

AUTO-CAPTURE

MEMORY_STORED

A debounced LLM extractor proposes TypedMemory facts after the run — shadow-only until calibrated.

GOVERN

MEMORY_SUPPRESSED • _CONSOLIDATED

Per-type budgets consolidate; panel rejects soft-suppress and drop from recall.



Durable, swappable, and off by default

Mem0MemoryBackend gives durable storage behind the hexagonal port — swap it without touching the loop. The recalled-memories panel and per-chat soft-suppress ship in the UI.

OFF BY DEFAULT

`memory_enabled = autocapture_enabled = False`

The input rail is a trained model, not a regex



Everyone ships “guardrails” now.



Most are a regex and a prayer — or an LLM judge on every request: slow, costly, and untested for over-blocking.

1 • PRECHECK

deterministic: length caps, injection regex, base64 decode, entropy check

zero cost • zero flake

2 • CLASSIFIER

fine-tuned DeBERTa-v3 with PiGuard MOF, exported to INT8 ONNX

reject ≥ 0.80 • accept ≤ 0.20

3 • NARROW JUDGE

small LLM, scoped to injection only — reached only when 1 & 2 defer

the expensive path, made rare



CI freezes the quality bar: malicious recall ≥ 0.95 , false-positive rate $< 2\%$ (Llama-Guard-3 $\approx 4\%$), over-defense measured on a held-out NotInject split — with a contamination guard that makes train/test leakage a hard error. **Training script included: `scripts/train_injection_classifier.py`.**

A goal judge with human-agreement receipts



Agents narrate success beautifully.



“Looks right to me” is not an eval. Judges that trust the agent's own story get gamed by fabricated progress.

$\kappa = 1.0$

rubric agreement

two blind annotators; gate ≥ 0.8
(docs/IAA)

$\alpha = 0.88$

gold-set pilot

Krippendorff's α on 43 items; 101-row
sheet at v0.9

16

failure codes

grounded-theory taxonomy across 5
axes

$\leq 5\%$

red-team flip rate

fabricated-progress attacks must not
flip verdicts



Verdicts are grounded in the tool trace, not the agent's narration — and the judge ships default-off, in shadow mode, until precision ≥ 0.90 and flip $\leq 5\%$ gates clear. **The full 7-stage methodology is in [.cursor/skills/llm-eval-grounded-theory/](#) — 62 tests cover the judge alone.**

Web search that costs \$0, needs no API key, and sanitizes what it reads



Every agent needs web search.



Search APIs meter every call, and raw snippets are an indirect-injection channel straight into your context window.

\$0

Self-hosted SearXNG sidecar — localhost or Cloud Run on port 8888, scale-to-zero, aggregating upstream engines. No vendor, no key, no meter.

Hexagonal port — WebSearchProvider protocol; SearXNG adapter + deterministic stub for CI (services/tools/search/)

Injection-sanitized — every title & snippet runs the retrieval rail before re-entering context; modifications logged

Fails terminal, not forever — dead backend → ok=False; no-progress detector (3 repeats, hard stop 5) ends loop-on-dead-tool

Tested failure-first — 15+ cases: HTTP 500, timeout, network error, empty results — written before the success paths

Verify: `docker compose -f docker-compose.searxng.yml up` · `WEB_SEARCH_PROVIDER=searxng`

Policy-gated deploys that idle at pocket change



“Production-grade” usually means a real cloud bill.



So agent projects stay on laptops — and the deployment lessons never get learned.

\$5–30/mo

dev tier, mostly free

Cloud Run scale-to-zero + Neon free
Postgres + Cloudflare Pages; the bill is
basically Cloudflare Pro

13 + 11

Rego + Gherkin gates

no plaintext secrets, no allUsers IAM,
SSE cache bypass — 108 conftest + 38
pytest checks, mutation-tested

4 gates

before any apply

tofu validate → tflint → conftest →
pytest; then BDD against the resolved
plan

A → D

graduation ladder

free tier → HA production; substrate
swaps touch only the composition
root, zero app code



Phased deploys (scripts/deploy_gcp.sh) with two human gates, digest-pinned images, CalVer+SHA deploy IDs — and a black-box → Langfuse relay with a dead-letter queue for observability.

40+ recipes: rebuild every piece yourself



Most repos document what; few teach how.



Cloning a black box teaches you nothing — and you can't maintain what you couldn't have built.

13

GCP deployment

adapters → cleanup, with cost
notes & rollback per step

9

Guardrails

the “five-door” security build,
precheck → classifier → judge

7

Governance

the flight-recorder: black box
→ Langfuse audit trails

3

Goal judge

grounded-theory failure
taxonomy → rubric → gold set

6+

TDD hardening

outcome correctness, schema
versioning, negative paths



Every recipe is dual-audience: exact commands for this repo, generic steps for your stack. Bonus: the reasoning prompts that structured this very deck — SCQA, Pyramid Principle, TDD-for-agents — ship in `research/`, runnable via `StructuredReasoning/cli_pyramid.py`.

Five things to steal in the next ten minutes

1

Run the architecture gates

```
pytest tests/architecture/ -q
```

watch 12 boundary tests defend the layers

2

Stand up free web search

```
docker compose -f docker-compose.searxng.yml up
```

then set `WEB_SEARCH_PROVIDER=searxng`

3

Train the injection classifier

```
python scripts/train_injection_classifier.py
```

DeBERTa-v3 + PIGuard MOF → INT8 ONNX

4

Read one recipe end-to-end

```
docs/recipes/guardrails/00_overview.md
```

the five-door security build

5

Borrow the reasoning prompts

```
research/pyramid_react_system_prompt.md
```

MECE issue trees + 8 validation checks, framework-agnostic

THE ANSWER, RESTATED

If this would've saved you a month, it's worth a star.

Architecture

12 CI gates

Guardrail ML

recall ≥ 0.95 gate

Goal-judge evals

$\kappa = 1.0 \cdot \alpha = 0.88$

Web search

\$0, no API key

GCP deploy

\$5-30/mo, policy-gated

Recipes

40+, dual-audience

Honest fine print: pre-1.0, single maintainer, LICENSE pending — until it lands, treat it as look-don't-ship and watch the repo for the license drop. ~20 live-LLM tests fail visibly in the local cache, excluded from CI on purpose (the recipes explain why). Every number here comes from a static audit of the repository.